

Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods

Gordon V. Cormack, Charles L. A. Clarke, Stefan Buettcher University of Waterloo

ACM SIGIR, 2009 • arXiv:No arXiv — SIGIR 2009 Proceedings pp.758-759

<https://arxiv.org/abs/No arXiv — SIGIR 2009 Proceedings pp.758-759>

Abstract

We present Reciprocal Rank Fusion (RRF), a simple rank aggregation method that consistently outperforms Condorcet fusing as well as individual Rank Learning methods. This finding is remarkable in that RRF requires no tuning parameters and requires very small amounts of training data in order to consistently outperform Condorcet Fusing, which requires no training data at all. Our experiments were conducted on TREC collections, using runs from various participating teams. The method computes a combined relevance score for each document using the reciprocal of each document's rank in each list: $RRF(d) = \sum 1/(k + r)$ where $k=60$ is a smoothing constant and r is the document's rank. Documents not appearing in a list receive no contribution from that list.

1. Introduction and Motivation

Rank aggregation is the problem of combining several ranked lists into a single consensus ranking. In information retrieval, this arises naturally when multiple retrieval systems produce different rankings of the same document set. The challenge is to combine these rankings in a way that is better than any individual system.

Previous approaches to rank fusion fall into two categories: (1) Score-based methods that combine the raw relevance scores from each system. These require score normalization since different retrieval systems produce scores on incompatible scales. (2) Rank-based methods that use only the relative ordering of documents. Reciprocal Rank Fusion belongs to this second category and is notably simple and effective.

2. The RRF Algorithm

2.1 Formula

Given multiple ranked lists $D_1, D_2, \dots, D_k$, the RRF score for document d is:

$$RRF_score(d) = \text{SUM over all rankers } r \text{ of: } 1 / (k + \text{rank}_r(d))$$

Where: k is a constant (Cormack et al. recommend $k=60$), and $\text{rank}_r(d)$ is the rank of document d in ranked list r (1-indexed). Documents not appearing in a list are simply not included in the sum for that list.

2.2 The $k=60$ Constant — Why This Value?

The constant $k=60$ was chosen empirically to be robust across different retrieval scenarios. Its effect is to reduce the impact of high rankings in a single system. Consider: if $k=0$, the top document in any list gets score 1.0 — a single strong signal dominates. With $k=60$, the top document gets only $1/61 \approx 0.016$, and the 10th ranked document gets $1/70 \approx 0.014$. This makes the fusion much more robust: a document ranked 1st in one list but absent from others does not dominate over a document ranked 5th in multiple lists.

Rank	k=0 Score	k=60 Score	Score Ratio (rank1/rank10)
1	1.000	0.01639	—
2	0.500	0.01613	1.016×
5	0.200	0.01538	1.065×
10	0.100	0.01429	1.147×
20	0.050	0.01250	1.311×
60	0.017	0.00833	1.967×

With $k=60$, the score ratio between rank 1 and rank 10 is only 1.15x, making the algorithm much less sensitive to exact rank positions. This is critical for combining systems with different ranking characteristics.

2.3 Worked Example

Suppose we have two ranked lists from a dense retriever and a BM25 system:

Document	Dense Rank	Sparse Rank	Dense Score	Sparse Score	RRF Score
doc_A	1	3	$1/(60+1)=0.01639$	$1/(60+3)=0.01563$	0.03202
doc_B	2	1	$1/(60+2)=0.01613$	$1/(60+1)=0.01639$	0.03252
doc_C	3	absent	$1/(60+3)=0.01563$	0	0.01563
doc_D	absent	2	0	$1/(60+2)=0.01613$	0.01613

Final ranking: doc_B (0.03252) > doc_A (0.03202) > doc_D (0.01613) > doc_C (0.01563). Notice doc_B wins because it appeared in BOTH lists, even though it was not ranked first in either. This is the key intuition: documents corroborated by multiple retrieval systems score higher.

3. Experimental Results

Cormack et al. evaluated RRF against several competing fusion methods on TREC data. The key finding was that RRF outperformed or matched all competitors despite its simplicity.

Fusion Method	Requires Score Normalization	Requires Tuning	Mean AP	Robustness
CombSUM (score-based)	YES	YES	Baseline	Low
CombMNZ	YES	YES	Baseline+2%	Medium
Borda Count	No	No	Baseline+1%	Medium

RRF (k=60)	No	No	Baseline+5%	High
Optimal Linear (oracle)	YES	YES (oracle)	Baseline+6%	N/A

RRF achieves near-oracle performance without score normalization or parameter tuning. This makes it immediately applicable to any combination of retrieval systems.

4. Why RRF Matters for Hybrid RAG Systems

In modern hybrid RAG architectures combining dense (semantic) and sparse (BM25/SPLADE) retrieval, RRF is the standard fusion method because:

1. No score normalization needed: Dense retrieval returns cosine distances (0 to 1); BM25 returns unbounded scores (0 to 20+). Linear combination requires careful normalization that varies per query. RRF sidesteps this entirely by using only ranks.
2. Robust to empty lists: If a query term is absent from all documents, BM25 returns no results. RRF handles this gracefully — documents only score from systems where they appear.
3. Corroboration bonus: Documents appearing in both dense and sparse results get boosted. This is exactly what you want — a document that is both semantically similar AND contains the right keywords should rank higher.
4. Production implementation: Qdrant implements RRF natively in its Query API (FusionQuery with Fusion.RRF). For pgvector, the SQL implementation uses CTEs with ROW_NUMBER() window functions.

5. Citation

Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2009), pp. 758–759.